

Báo cáo kiểm toán sử dụng AI (AI Audit Report) — Biểu mẫu 5 phần theo từng Artifact

Phụ lục bắt buộc cho mọi bài tập có ứng dụng AI (HW#01–HW#06, và Seminar).

Chuyển thể từ Med Kharbach, PhD (2026) — AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0. Bản chuyển thể này phục vụ cho môn học CS423 / CSC13003 Kiểm thử phần mềm tại FIT@HCMUS.

1. Thông tin sinh viên

Trường thông tin	Giá trị
Họ và tên sinh viên:	Phan Quốc Thịnh
Mã số sinh viên (MSSV):	23127486
Lớp / Khóa:	(Điền thông tin lớp/khóa)
Mã bài tập (Assignment ID):	HW05
Ngày thực hiện:	15/08/2026
Công cụ AI sử dụng:	Claude Sonnet 4.6 (thông qua Antigravity IDE)
Có sử dụng AI không?	☒ Có ☐ Không

2. Hướng dẫn thực hiện (Đọc kỹ trước khi điền)

- Thêm một hàng cho mỗi sản phẩm (artifact) do AI hỗ trợ sinh ra (test case, script, checklist, OpenAPI spec, JMeter test plan, báo cáo phân tích,...).
- Dán nguyên văn prompt đã gửi cho AI (Verbatim prompt) — TUYỆT ĐỐI KHÔNG diễn giải lại theo ý mình.
- Dán nguyên văn phản hồi của AI (hoặc đính kèm ảnh chụp màn hình có chú thích trong báo cáo).
- Gán nhãn đánh giá chất lượng: **HỢP LỆ (VALID)** / **KHÔNG HỢP LỆ (INVALID)** / **CHƯA ĐẦY ĐỦ (INCOMPLETE)**.
- Phần giải thích lý do phải trích dẫn slide bài giảng, điều khoản ISTQB hoặc tiêu chuẩn kỹ thuật RFC.
- Nêu rõ phần chỉnh sửa, bổ sung của sinh viên đối với artifact đó.

3. Bảng kiểm toán chi tiết (Audit Table — một hàng cho mỗi artifact)

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
Công cụ: Claude Sonnet 4.6 Thời gian: 20:37 15/08/2026 Prompt: dựa vào đề bài trong @[:\Users\Public\Projects\Testing_HCMUS\HW5\docs\2026.HW05.Performance Testing_En (1).md] , hãy tạo tất cả các file cần thiết (...) đưa vào thư mục @[:\Users\Public\Projects\Testing_HCMUS\HW5\eshop-sut\submissions] . biết tôi tên Phan Quốc Thịnh, MSSV: 23127486	Đã đọc xong đề bài. Tôi sẽ tạo các file skeleton cần thiết cho bài nộp trong thư mục <code>submissions</code>. Dựa vào đề bài, các file cần thiết bao gồm: 1. Report chính (Markdown); 2. AI Audit Report; 3. AI Critique (Markdown); 4. README.md; 5. Git commit log (text file); 6. 3 test plans (JMeter <code>.jmx</code> files); 7. CSV data files; 8. 3 raw .jtl log files; 9. Bug report (Markdown); 10. Continuous Performance Testing proposal	CHƯA ĐẦY ĐỦ (INCOMPLETE)	AI tự động sinh cả cá thực thi test (<code>jmx</code>, <code>jtl</code>), chưa thiết kế kịch bản tiết và chưa có sự xác nhận luồng nghiệp vụ người dùng. Theo IST FL §1.4 (Quy trình kiểm thử), giai đoạn chuẩn tài liệu cấu trúc (Test Planning/Documenta phải tách biệt với giai đoạn sinh artifact thực thi.

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
Công cụ: Claude Sonnet 4.6 Thời gian: 20:43 15/08/2026 Prompt: tiếp tục đi, nhưng các file được tạo trong quá trình thực thi test như jtl, test plan, html reports thì chưa cần tạo đâu, chỉ cần tạo các file tài liệu md cần thiết thôi	Xong! Tất cả các file tài liệu <code>.md</code> đã được tạo/cập nhật. Các file: <code>README.md</code> (bảng self-assessment + test summary report ☒ Mới); <code>Report.md</code> (Report chính ☒ Mới); <code>AI_Audit.md</code> (đã điền tên + MSSV ☒ Cập nhật); <code>AI_Critique.md</code> (AI Critique 200-300 từ ☒ Mới); <code>bug_report.md</code> (Bảo cáo bug/performance issues ☒ Mới); <code>git_commit_log.txt</code> (Git commit log ☒ Mới).	HỢP LỆ (VALID)	Đáp ứng chính xác yêu cầu tạo đầy đủ các tài liệu báo cáo (.md) theo đúng quy định bài của HW05 (Mục 1
Công cụ: Claude Sonnet 4.6 Thời gian: 20:47 15/08/2026 Prompt: okay, hãy dựa vào yêu cầu đề bài HW5 (...), rồi tạo các skill cần thiết cho bài tập này nhé. lưu ý: task 1 có được giảng viên note lại như sau: you now need to perform load, stress, and spike testing on a single workflow that covers three API endpoint groups.	Tất cả 4 skills đã được tạo đúng cấu trúc: <code>hw5-test-plan-design</code> (Khám phá SUT API → thiết kế workflow E2E → sinh 3 file .jmx + CSV → cập nhật Report.md Sec 3 & 4.1-4.2, Task 1); <code>hw5-jtl-analysis</code> (Parse jtl logs → tính metrics → prompt AI theo 4 bước → scaffold Report.md Sec 5.1-5.3, Task 2); <code>hw5-ci-perf-proposal</code> (Thiết kế pipeline CI/CD → sinh Mermaid flowchart → bản luận trade-off → cập nhật Report.md Sec 6, Task 3); <code>hw5-ai-audit-update</code> (Ghi verbatim prompt + AI output + verdict vào AI_Audit.md, Mọi task).	CHƯA ĐẦY ĐỦ (INCOMPLETE)	Trong <code>hw5-test-plan-design</code>, AI tự gán sẵn endpoint ví dụ và tự nghĩ nghĩa cấu trúc cột CSV mà không dừng lại ý cầu người dùng chỉ định workflow cụ thể. Vì nguyên tắc phân công giữa các thành viên (1 5: "no two members test the same workflow"
Công cụ: Claude Sonnet 4.6 Thời gian: 22:42 15/08/2026 Prompt: hãy sử dụng /performance-test-plan-design để lên plan cho kiểm thử hiệu năng, biết tôi lựa chọn kiểm thử trên luồng: Login → search + detail → cart → áp mã giảm giá → checkout. Mapping 3 nhóm: Auth-heavy: POST /api/login; Read-heavy: GET /api/products?search= → GET /api/products/{id}; Transactional: POST /api/cart → POST /api/apply-coupon → POST /api/checkout. Coupon chọn VIP100 (max 2 lần/user, ngưỡng 300K). Backend check total_amount > min_order_amount (strict >).	Plan đã được tạo. Tóm tắt những điểm cần bạn xác nhận trước khi tôi bắt đầu sinh file: 3 Open Questions: Q1: Backend có thực sự nhận <code>total_amount</code> từ client trong <code>POST /api/checkout</code> không? (Default: Có → dùng <code>\${final_amount}</code>); Q2: <code>POST /api/apply-coupon</code> KHÔNG cần header <code>Authorization</code>? (Default: Đúng → bỏ Bearer ở step 5). Q3: Coupon nào làm mặc định cho CSV? (Default: <code>VIP100</code>). Những gì tôi sẽ sinh ra: 1. <code>test_data.csv</code> – 20 dòng, 9 cột; 2. <code>23127486_Load_20260815.jmx</code> – 20 VU, 300s; 3. <code>23127486_Stress_20260815.jmx</code> – Step-up 50→200 VU; 4. <code>23127486_Spike_20260815.jmx</code> – 5→100→5 VU; 5. <code>Report.md</code> – Cập nhật Section 3.1, 3.2, 4.1.x, 4.2	HỢP LỆ (VALID)	Thiết kế implementat plan đầy đủ theo đứ Step 1 của SKILL.md; động phát hiện các rủi ro kỹ thuật (backend nhận <code>total_amount</code> từ client, coupon limit, r auth apply-coupon) và tạo checkpoint yêu cầu người dùng xác nhận trước khi sinh artifact (ISTQB FL §1.4).

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (ISTQB / Chuẩn kỹ thuật)
Công cụ: Claude Sonnet 4.6 Thời gian: 22:51 15/08/2026 Prompt: Comments on artifact URI: file:///c%3A/Users/thinh/.gemini/antigravity-ide/brain/5d04719c-95a9-40bb-8356-6032841e2465/implementation_plan.md The user has approved this document.	☑ Hoàn thành – Task 1 Performance Test Plan Design. Tất cả 5 file đã được tạo trong <code>submissions/: test_data.csv</code> (20 rows × 9 cols, coupon VIP100, total_before>300K ☑); <code>23127486_Load_20260815.jmx</code> (20 VU, 60s ramp, 300s — View Results Tree ☑); <code>23127486_Stress_20260815.jmx</code> (Step-up 50→100→150→200 VU — Summary Report ☑); <code>23127486_Spike_20260815.jmx</code> (5→100→5 VU, ramp spike 5s — Aggregate Report ☑); <code>Report.md</code> (Section 3.1, 3.2, 4.1.1–4.1.3, 4.2 đã điền ☑).	HỢP LỆ (VALID)	Sinh thành công toàn bộ test plans (.jmx) cho kịch bản Load, Stress, Spike với các listener khác nhau, file <code>test_data.csv</code> với 20 dữ liệu thiết kế trong <code>Report.md</code> theo đúng định Task 1 và chuẩn thuật JMeter Test Plan (ISTQB FL §4.1).
Công cụ: Claude Sonnet 4.6 Thời gian: 09:22 16/08/2026 Prompt 2a — Thiết lập ngữ cảnh & Phân tích tổng thể: I am performing performance testing on an EShop REST API using JMeter. The SUT is a Vietnamese e-commerce demo application with SQLite database. I have three test scenarios: Load (normal traffic), Stress (above-normal until breaking point), and Spike (sudden burst). Here are the raw metrics I extracted from the .jtl logs: Load Test (20 VU, 300 s, 60 s ramp-up): Total Requests=8,040; Error Count=0; Error Rate=0.00%; Throughput=26.97 RPS; Avg=2.7 ms; Median=2.0 ms; P90=6.0 ms; P95=7.0 ms; P99=10.0 ms; Min/Max=0/37 ms; Duration=298.1 s. Stress Test (50→200 VU stepped, 569 s): Total Requests=155,778; Error Count=0; Error Rate=0.00%; Throughput=273.78 RPS; Avg=5.5 ms; Median=4.0 ms; P90=12.0 ms; P95=17.0 ms; P99=28.0 ms; Min/Max=0/140 ms; Duration=569.0 s. Spike Test (5→100→5 VU, near-instant spike, 148 s): Total Requests=19,719; Error Count=0; Error Rate=0.00%; Throughput=133.23 RPS; Avg=140.0 ms; Median=142.0 ms; P90=228.0 ms; P95=253.0 ms; P99=300.8 ms; Min/Max=0/377 ms; Duration=148.0 s. Please analyse these results and tell me: 1. Is the system's performance acceptable under Load conditions? 2. At what point does it degrade under Stress? 3. How does it recover from the Spike?	1. Performance under Load (20 VU): The system performs excellently under normal load conditions. With only 20 virtual users and a 60-second ramp-up, the average response time of 2.7 ms and median of 2.0 ms are remarkably low — well below any industry threshold for acceptable REST API performance (typically 200–500 ms for e-commerce). The P95 of 7 ms and P99 of 10 ms confirm extremely low tail latency. The zero error rate across 8,040 requests indicates complete stability. Throughput of ~27 RPS is consistent with the 20-VU × workflow cycle time (including think times of 1.5 s + 1.5 s + 1.0 s = 4 s per cycle plus response time). Conclusion: Performance is fully acceptable under Load — the system handles normal traffic with significant headroom. 2. Degradation under Stress (50→200 VU): The system surprisingly shows no errors even at 200 VU, and response times remain usable. However, there is measurable degradation compared to Load: Average response time increased 2.0× (2.7 ms → 5.5 ms); P95 increased 2.4× (7.0 ms → 17.0 ms); P99 increased 2.8× (10.0 ms → 28.0 ms); Max response time jumped from 37 ms → 140 ms. The throughput scaling to 273.78 RPS (10× the Load RPS) with 10× the concurrent users suggests near-linear scaling without a clear breaking point. This may indicate: (a) the SQLite WAL mode handles concurrent reads well, (b) the step-up design (with 60 s sustain per phase) gave the system time to stabilize, or (c)	HỢP LỆ (VALID)	AI phân tích đúng xu hướng tổng thể: Load Stress chưa đến điểm (breaking point), Spike gây độ trễ tăng mạnh nhưng không xuất hiện lỗi HTTP. Kết luận hợp với kết quả thực từ log JTL. Tuy nhiên một số điểm suy diễn quá đà cần human review (đã đối chiếu trong IV 5.2). Đánh giá tổng thể phù hợp ISTQB PT §3 (Phân tích kết quả kiểm thử hiệu năng).

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
	the Node.js event loop is not yet CPU-saturated at 200 VU on this hardware. No clear breaking point was observed, but latency degradation begins to show at the P99 level (28 ms) — the stress ceiling appears to be beyond 200 VU for this test configuration. 3. Recovery from Spike (5→100→5 VU): The Spike test reveals the most interesting results. During the spike phase (100 VU, near-instant 5 s ramp-up): Average response time spiked dramatically to 140 ms (52× higher than Load); P95 reached 253 ms, P99 reached 301 ms, Max hit 377 ms; Error rate remained 0.00% — no requests failed despite the sudden 20× load increase. The spike metrics are dominated by the spike phase itself (the majority of the 148-second test). The <code>POST /api/apply-coupon</code> was the slowest endpoint at avg 208.8 ms / P95 306 ms during the spike — likely because it performs DB reads (coupon validation) and is serialized by SQLite's single-writer lock under high concurrency. Recovery behavior cannot be isolated from this data because the JTL metrics are aggregated across all three sub-phases (baseline/spike/recovery). A per-phase breakdown would require timestamp filtering.		

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
Công cụ: Claude Sonnet 4.6 Thời gian: 09:22 16/08/2026 Prompt 2b — Đề xuất các ngưỡng hiệu năng: Based on the metrics above, suggest appropriate performance thresholds for this EShop API: - Maximum acceptable average response time (ms) - Maximum acceptable 95th percentile response time (ms) - Maximum acceptable error rate (%) - Recommended maximum concurrent users before degradation. Justify each threshold value with industry standards or the SUT's observed behavior.	Based on the observed metrics and industry standards for a small Vietnamese e-commerce application: Max acceptable average response time: 50 ms (Load test avg = 2.7 ms; Stress avg = 5.5 ms. A threshold of 50 ms gives 18× headroom from the Load baseline and 9× from the Stress baseline, consistent with Google's "200 ms feels instant" guideline. For backend API (not full page load), 50 ms is a strict but realistic bound.); Max acceptable P95 response time: 200 ms (Load P95 = 7 ms; Stress P95 = 17 ms; Spike P95 = 253 ms. The Spike P95 (253 ms) exceeded this threshold, indicating the current SUT cannot meet this bound under a 20× spike. 200 ms aligns with APDEX "satisfied" threshold (T=0.2 s) for API services.); Max acceptable error rate: 1% (All three scenarios showed 0.00% error rate. The 1% threshold is the standard SLA ceiling for e-commerce APIs (ISTQB Performance Testing Foundation). For transactional endpoints (checkout), a stricter bound of 0.1% is recommended.); Max concurrent users before degradation: 150 VU (At 150 VU (Phase 3 of Stress), P99 = 28 ms and max = 140 ms — still within acceptable bounds. Beyond 150 VU, tail latency begins compounding. However, since no error rate was observed at 200 VU, the "soft" degradation point is estimated at ~150 VU for latency-sensitive SLAs.); Additional: Maximum P99 response time: 500 ms (the spike showed P99 = 301 ms, barely under this bound); Minimum throughput: 25 RPS sustained under Load (current: 26.97 RPS — acceptable).	CHƯA ĐẦY ĐỦ (INCOMPLETE)	AI đề xuất các ngưỡng khá hợp lý nhưng khi định "At 150 VU (Phase of Stress), P99 = 28 ms" là không chính xác — số P99 = 28 ms là giá tổng hợp (aggregate) toàn bộ bài Stress test, không phải riêng Phase AI không có số liệu rõ ràng theo phase nhưng tự suy diễn. Theo ISTQB PT §3.1, việc thiết lập ngưỡng phải dựa trên liệu đo đạc thực tế, không dựa vào suy đ
Công cụ: Claude Sonnet 4.6 Thời gian: 09:22 16/08/2026 Prompt 2c — Nhận diện điểm nghẽn hiệu năng (Bottleneck): Looking at the per-endpoint breakdown from the .jtl data: [Dữ liệu chi tiết Load, Stress, Spike theo từng endpoint] Which endpoints are the bottlenecks? Which HTTP response codes appear most frequently during stress/spike? What do you think causes the degradation?	Primary Bottleneck: POST /api/checkout (Step 6) Across all three scenarios, POST /api/checkout consistently has the highest response time among all endpoints: Load Avg 6.9 ms P95 11 ms — 4.6× slower than POST /api/cart; Stress Avg 10.7 ms P95 24 ms — 4.9× slower than POST /api/cart; Spike Avg 148.4 ms P95 229 ms. This endpoint performs the most DB writes in a single transaction: creating an order record, updating stock quantities, and deleting cart items. SQLite's single-writer model means all concurrent checkout	HỢP LỆ (VALID)	AI nhận diện chính xác hai điểm nghẽn lớn nhất (checkout và apply-coupon), đồng thời gợi thích hợp lý cơ chế nghẽn hàng đợi (queue serialization) của Node event loop dưới tải đa biến. Mặc dù có một suy đoán sai về mã nguồn thực tế (nhầm driver, tưởng tượng bước DB của checkout), phân tích logic phù hợp ISTQB PT §3.3 (Xác định điểm nghẽn hiệu năng

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
	requests must queue at the write lock. Secondary Bottleneck: POST /api/apply-coupon (Step 5) During the Spike, POST /api/apply-coupon becomes the worst endpoint at avg 208.8 ms / P95 306 ms — even slower than checkout. This endpoint reads the coupon record, checks validity, counts usage per user, and returns the final amount. Under high concurrency, these multiple sequential DB reads on a SQLite database with no query caching create significant serialization delays. Surprising non-bottleneck: POST /api/cart (Step 4) Despite being a write operation, POST /api/cart has the lowest response times in all scenarios (Load: 1.5 ms, Stress: 2.2 ms, Spike: 65.4 ms). This is likely because the cart insert is a simple single-row write without complex joins or aggregations. Uniform spike impact on GET endpoints: The GET /api/products?search= and GET /api/products/{id} show nearly identical avg response times during the Spike (~131–144 ms). This uniformity suggests the bottleneck is not query complexity but rather the Node.js event loop saturation under 100 concurrent VUs — all requests queue behind the single-threaded event loop regardless of query type. Root causes of degradation: SQLite single-writer lock: All write operations compete for the same write lock. As concurrency increases, write queuing dominates response time. No connection pooling for SQLite: Node.js + better-sqlite3 uses a single synchronous DB connection. All requests are serialized at the DB layer. No caching layer: Products, coupons, and user data are read from SQLite on every request. A Redis cache would eliminate most read latency under spike conditions. Sequential multi-query operations: apply-coupon performs 3–4 sequential DB queries — each query holds the DB open longer, increasing contention.		

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
	HTTP response codes: Since error rate = 0% across all scenarios, only HTTP 200 and 201 were observed. No 4xx/5xx errors were triggered, meaning the SQLite serialization caused latency but not failures at these concurrency levels.		
Công cụ: Claude Sonnet 4.6 Thời gian: 09:22 16/08/2026 Prompt 2d — Đề xuất phương án tối ưu hóa: Given the bottlenecks you identified, propose 5-8 specific optimizations that could improve performance. For each optimization, describe: - What it is - How it addresses the bottleneck - Whether it requires application code changes, database changes, infrastructure changes, or JMeter configuration changes	Here are 7 specific optimizations for the EShop SUT, ordered by estimated impact: Optimization 1: Migrate from SQLite to PostgreSQL — What: Replace SQLite with PostgreSQL (or MySQL) as the backend database. How it addresses the bottleneck: PostgreSQL supports true concurrent writes via MVCC (Multi-Version Concurrency Control), eliminating the single-writer lock that serializes all write operations in SQLite. Under 200 VU, checkout requests would no longer queue sequentially. Change type: Database + Application code (connection string, query syntax, driver <code>pg</code> vs <code>better-sqlite3</code>). Optimization 2: Add Redis Caching for Products and Coupons — What: Introduce a Redis in-memory cache for product data (<code>GET /api/products</code>, <code>GET /api/products/{id}</code>) and coupon metadata (<code>POST /api/apply-coupon</code> read phase). How it addresses the bottleneck: Eliminates SQLite read queries for hot data. Product catalog and coupon definitions rarely change — cache TTL of 60 s would serve 95%+ of read requests from memory. During the Spike, <code>apply-coupon</code> avg would drop from 208 ms to ~2-5 ms for the read phase. Change type: Infrastructure (Redis server) + Application code (cache-aside pattern in route handlers). Optimization 3: Implement Database Connection Pooling — What: Replace the single <code>better-sqlite3</code> synchronous connection with a pooled async driver (e.g., <code>sqlite3</code> + <code>generic-pool</code>, or migrate to PostgreSQL with <code>pg-pool</code>). How it addresses the bottleneck: Allows Node.js to handle multiple DB queries concurrently instead of serializing them through the event loop. Reduces the "idle waiting" time when one query holds the connection. Change type: Application code only.	CHƯA ĐẦY ĐỦ (INCOMPLETE) AI đề xuất 7 giải pháp, tuy nhiên có một điểm kỹ thuật không chính xác cần kiểm d (Human review): (1) E xuất 3 — driver <code>sqlite</code> vốn đã là async, việc pool không mang lại ích rõ rệt cho SQLite đơn; (2) Đề xuất 5 — kiểm <code>LIKE '%keywor</code> có ký tự đại diện ở đầ không thể tận dụng <code>tree index</code>; (3) Đề xuất — <code>PRAGMA read_uncommitted=</code> chỉ có tác dụng trong shared-cache mode, không áp dụng cho c hình mặc định. Theo ISTQB PT §4.1, các đề xuất cải thiện hiệu nă phải được kiểm chún tính khả thi kỹ thuật trước khi áp dụng.	

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
	Optimization 4: Batch/Optimize the Checkout Transaction — What: Rewrite the <code>POST /api/checkout</code> handler to execute all DB operations (insert order, update stock, delete cart) in a single SQLite transaction using prepared statements. How it addresses the bottleneck: Reduces the number of round-trips to SQLite from multiple sequential calls to one atomic transaction. SQLite's WAL mode handles single transactions efficiently, so batching them reduces lock-hold time per request. Change type: Application code only. Optimization 5: Add Database Indexes — What: Add composite indexes on frequently queried columns: <code>users(email)</code>, <code>products(name)</code> (for LIKE search), <code>coupon_usage(user_id, coupon_id)</code>, <code>orders(user_id)</code>. How it addresses the bottleneck: The <code>GET /api/products?search=</code> endpoint likely performs a full-table LIKE scan on the <code>name</code> column. Under Stress, 26,124 search requests × full-table scan = significant I/O. An index reduces this to a B-tree seek. Similarly, coupon usage count query on <code>coupon_usage</code> table is O(n) without an index. Change type: Database only (SQL <code>CREATE INDEX</code> statements). Optimization 6: Implement API Response Compression (gzip) — What: Enable gzip compression in the Express.js server using the <code>compression</code> middleware. How it addresses the bottleneck: Reduces the bytes transferred per response, particularly for <code>GET /api/products?search=</code> which returns a JSON array. Under high concurrency, reduced payload size decreases network I/O wait and allows the event loop to process more requests per second. Change type: Application code only (one-line <code>app.use(compression())</code>). Optimization 7: Separate Read and Write Paths (CQRS lite) — What: Configure SQLite with <code>journal_mode=WAL</code> and <code>PRAGMA read_uncommitted=1</code> for read-only operations, while keeping strict ACID for writes. Alternatively, introduce a read replica (even a file copy with <code>rsync</code>) for GET		

(1) Prompt & Công cụ	(2) Phản hồi của AI (Verbatim)	(3) Đánh giá	(4) Cơ sở lý luận (IS / Chuẩn kỹ thuật)
Công cụ: Claude Sonnet 4.6 (Antigravity IDE) Thời gian: 22:21 16/08/2026 Prompt (Task 3 — Đề xuất Continuous Performance Testing tích hợp CI/CD): sử dụng /ci-performance-proposal để hoàn thành task 3 của HW5	endpoints. How it addresses the bottleneck: SQLite WAL mode already allows concurrent reads alongside writes, but <code>read_uncommitted</code> mode allows readers to bypass the shared lock entirely. This would improve <code>GET /api/products</code> and <code>GET /api/products/{id}</code> latency under spike conditions. Change type: Application code (PRAGMA settings at DB open time) + minor infrastructure.		
	AI đã đọc SKILL.md của <code>ci-performance-proposal</code> , khám phá cấu trúc SUT (Node.js Express + SQLite, GitHub Actions, không có Dockerfile), và sinh toàn bộ nội dung Mục 6 (Task 3) và Mục 8 (Kết luận) trong Report.md. Nội dung bao gồm: (1) Mục 6.1 — Bản tường trình 3 đoạn mô tả mô hình CPT kích hoạt có chọn lọc trên các commit thay đổi backend, chạy smoke test JMeter CLI 5–10 VU trong 60–120s, ngưỡng hồi quy p95 là 20%, quản lý baseline tự động; (2) Mục 6.2 — Sơ đồ Mermaid flowchart 11 node thể hiện luồng quyết định CI/CD; (3) Mục 6.3 — Bảng so sánh Chi phí vs Độ bao phủ (4 phương án), Bảng Trade-off về Cảnh báo sai (False Alarm - 5 loại lỗi kèm giải pháp), Các điểm lưu ý đặc thù của EShop (SQLite concurrency, cold-start bias, cấu hình môi trường); (4) Mục 8 — Đoạn kết luận tổng kết toàn bộ 3 task.	HỢP LỆ (VALID)	AI tuân thủ chính xác định trong SKILL.md: kế quy trình pipeline đủ 5 thành phần cốt (cơ chế trigger, smoke test, quản lý baseline phát hiện hồi quy, báo cáo kết quả), sơ đồ Mermaid chuẩn cú pháp phân tích trade-off sắc. Nội dung được t chỉnh sát với hiện trạng của EShop SUT. Phù hợp tiêu chí mức G9.6 Dis (ISTQB PT §5: Đề xuất tiến quy trình kiểm th

4. Tổng kết mức độ chính xác của AI (Summary of AI Accuracy)

Tổng hợp các đánh giá từ Mục 3 và hoàn thiện bảng thống kê dưới đây:

Chi số đánh giá	Số lượng artifact	Tỷ lệ phần trăm (%)
Tổng số artifact do AI sinh ra được kiểm toán	11	100%
HỢP LỆ (VALID — chính xác, chấp nhận nguyên trạng)	7	63.6%
KHÔNG HỢP LỆ (INVALID — sai sót, bị bác bỏ hoàn toàn)	0	0%
CHƯA ĐẦY ĐỦ (INCOMPLETE — chấp nhận sau khi người dùng chỉnh sửa/bổ sung)	4	36.4%

5. Kết luận — Khi nào nên (hoặc không nên) sử dụng AI?

Qua toàn bộ quá trình thực hiện bài tập HW05, AI (Claude Sonnet 4.6) thể hiện thế mạnh vượt trội trong việc nhanh chóng phân tích tài liệu đặc tả, tạo cấu trúc khung (scaffolding) cho các báo cáo kỹ thuật, xây dựng các bộ Agent Skill có khả năng tái sử dụng cao và sinh toàn bộ các kịch bản kiểm thử hiệu năng JMeter (jmx, dữ liệu CSV tham số hóa) bám sát luồng nghiệp vụ thực tế. Khi được hướng dẫn bởi quy trình có ranh giới rõ ràng và có sự kiểm soát của con người (Human-in-the-Loop thông qua Implementation Plan), AI thiết kế rất chuẩn xác các tham số kiểm thử cho từng kịch bản (Load, Stress, Spike), sử dụng đúng Listener và trích xuất dữ liệu động chính xác bằng JSON Extractor.

Tuy nhiên, AI thường bộc lộ điểm yếu khi phân tích sâu kiến trúc mã nguồn và suy đoán nguyên nhân gốc rễ (root cause) của hiệu năng. AI có xu hướng ngoại suy theo các mẫu thiết kế phổ biến dẫn đến ảo giác (hallucination) về cấu hình SQLite WAL mode, nhầm lẫn driver cơ sở dữ liệu và hiểu sai cấu trúc dữ liệu log dòng thời gian của JMeter. Khuyến nghị: Cần duy trì mô hình AI-Augmented Testing có kiểm soát, trong đó AI đảm nhận vai trò tăng tốc độ sinh artifact và khung phân tích, còn kỹ sư kiểm thử luôn phải trực tiếp đối chiếu, thẩm định mã nguồn và dữ liệu thực nghiệm trước khi đưa ra quyết định kỹ thuật cuối cùng.

6. Công bố bắt buộc về việc sử dụng AI (Mandatory Disclosure)

"Khung cấu trúc bài nộp, các mẫu báo cáo, bộ kỹ năng Agent Skills, các kịch bản kiểm thử hiệu năng JMeter (Load, Stress, Spike) và tập dữ liệu kiểm thử CSV ban đầu được khởi tạo với sự hỗ trợ của mô hình Claude Sonnet 4.6 (thông qua Antigravity IDE); tôi đã trực tiếp kiểm tra, điều chỉnh kỹ năng thiết kế kịch bản để bắt buộc xác nhận luồng nghiệp vụ do người dùng chỉ định, phê duyệt quy trình giao dịch E2E 6 bước có áp dụng mã giảm giá, đồng thời xác minh toàn bộ các tệp kịch bản .jmx và tài liệu kỹ thuật liên quan. Đối với Task 2, tôi sử dụng Claude Sonnet 4.6 để phân tích dữ liệu log thô .jtl thông qua 4 prompt có cấu trúc (thiết lập ngữ cảnh/phân tích tổng thể, đề xuất ngưỡng hiệu năng, nhận diện điểm nghẽn, và khuyến nghị tối ưu hóa); các phản hồi của AI được ghi lại nguyên văn trong Mục 5.1 của Report.md và AI_Audit.md; quá trình săn lỗi suy diễn sai (Mục 5.2) và đánh giá tính khả thi kỹ thuật (Mục 5.3) do chính tôi trực tiếp đối chiếu với mã nguồn và dữ liệu thực tế. Đối với Task 3, tôi sử dụng Claude Sonnet 4.6 (kỹ năng ci-performance-proposal) để sinh bản đề xuất hoàn chỉnh về Continuous Performance Testing (Mục 6) và Kết luận (Mục 8) trong Report.md; tôi đã rà soát và xác nhận cơ chế trigger, ngưỡng hồi quy và các lưu ý đặc thù phù hợp với kiến trúc thực tế của SUT. Báo cáo kiểm toán AI chi tiết được đính kèm tại Phụ lục A. Tôi cam kết không sử dụng AI để sinh bất kỳ thành phần nào nằm trong danh mục bị nghiêm cấm."

7. Chữ ký xác nhận

Họ và tên sinh viên (in hoa):	Phan Quốc Thịnh
Mã số sinh viên (MSSV):	23127486
Lớp / Khóa:	(Điền thông tin lớp/khóa)
Môn học:	CS423 / CSC13003 – Kiểm thử phần mềm
Giảng viên phụ trách:	TS. Lâm Quang Vũ / TS. Trần Duy Hoàng
Ngày xác nhận:	15/08/2026
Chữ ký:	(Ký tên tại đây)

Tài liệu tham khảo

- Kharbach, M. (2026). AI Use Policy Templates for Higher Education. CC BY-NC-SA 4.0.
- Giáo trình và Chuẩn kiểm thử ISTQB Foundation Level & Performance Testing Extension.
- Hardman, P. (2025). A Post-AI Learning Taxonomy.
- Fuster Rabella, M. (2025). OECD Education Working Paper No. 338.
- Perkins, M., Roe, J., & Furze, L. (2025). AI Assessment Scale.
- Anthropic (2025). Building Reliable AI Test Agents — Engineering Blog.
- DeepEval & Promptfoo Documentation — Frameworks kiểm thử và đánh giá hệ thống LLM.